Name: Francis Velasco

NetID: fiv3098

Email: francisvelasco2025@u.northwestern.edu

Technical Skills:

COMPONENTS THINKING

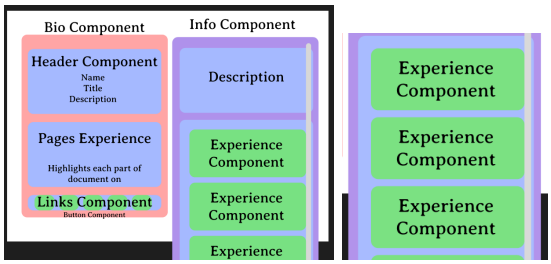
In both Project 1 and Project 2, I emphasized component thinking by drafting initial mockups and determining the larger organisms in which I could break into smaller components. For example, I broke my bio component into a header and imgButtons for my socials in Project 1. I also created Experience and Project components to reuse in an experience/achievements list. In Project 2, I built commentSection using smaller comment components as well as newsfeedList using smaller newsfeedCard components. Throughout both projects I tended to create components for either component thinking or for practicing good module design pattern.

FROM project-1-personal-portfolio-Dragoon788/blob/main/my-app/src/app/page.tsx

(images are also clickable)

Code Artifact #1:

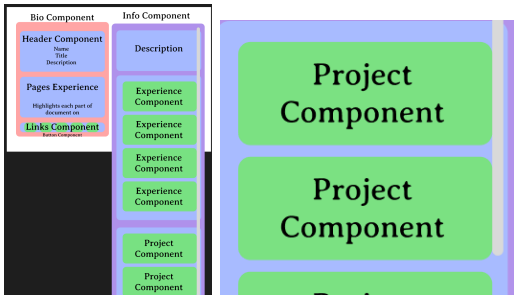
```
30            <div className="flex flex-col">
31              <Experience startDate="July 2022" endDate="Present" position="Student" des="asdfasdfjawejfjasdfjasdjfajwejfjae"/>
32              <Experience startDate="July 2022" endDate="Present" position="Student" des="asdfasdfjawejfjasdfjasdjfajwejfjae"/>
33              <Experience startDate="July 2022" endDate="Present" position="Student" des="asdfasdfjawejfjasdfjasdjfajwejfjae"/>
```



In Project 1, I created Experience card components at could be reused for each set of experience that I would want to display on my resume. The fields are main aspects important in describing and displaying that experience. I designed the site layout and components in Figma before working on the code in order to decide what that experience list would look like on the page.

Code Artifact #2:

```
40            <div className="flex flex-col">
41              <Proj projName="Web App" des="asdfasdfjawejfjasdfjasdjfajwejfjae" src="Kaibigan1.jpg"/>
42              <Proj projName="Web App" des="asdfasdfjawejfjasdfjasdjfajwejfjae" src="Kaibigan1.jpg"/>
43              <Proj projName="Web App" des="asdfasdfjawejfjasdfjasdjfajwejfjae" src="Kaibigan1.jpg"/>
```

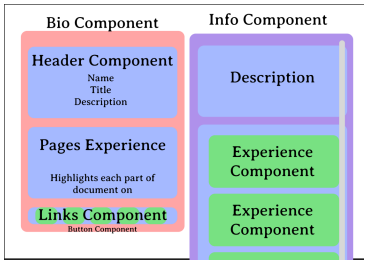


In Project 1, I also made Project card components that could be reused for the list of projects that I want to show in my portfolio. The fields are also main parts of a project that are important to describing and displaying an individual project. The Figma design allowed me to layout the project list before coding the site.

FROM project-1-personal-portfolio-Dragoon788/blob/main/my-app/src/app/components/bio.tsx

Code Artifact #3:

```
23            <footer className="flex flex-row justify-start">
24                <Button src="githubicon.png" srcName="Github" link="https://github.com/Dragoon788"/>
25                <Button src="LinkedInicon.png" srcName="LinkedIn" link="https://www.linkedin.com/in/francis-velasco-a53ba8242/"/>
26            </footer>
```



Lastly in Project 1, I also made sure to include reusable image button components for when an individual would like to visit my different social accounts. The Figma design shows the green imgButton components in a larger bio component.

FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/CommentSection.tsx

Code Artifact #4:

```
165           {/* Comments List */}
166           <div className="space-y-4">
167             {data?.postComments?.map((comment) => (
168               <Comment key={comment.id} comment={comment} />
169             ))}
```

In Project 2, I fetched post comments from the backend and mapped each comment to a comment component to display on the frontend. This design emphasized the inherent reusability of comments in a comment section

FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/NewsfeedList.tsx

Code Artifact #5:

```
133             {posts.map((post: Post) => (
134               <NewsfeedCard
135                 key={post.id}
136                 id={post.id}
```

In Project 2 – similar to the previous code artifact – I fetched all the posts and mapped each post to a newsfeedCard component to display on the homepage. Like the comment section, the NewsfeedList reuses the smaller NewsfeedCard components in its design.

ARCHITECTURE/DESIGN PATTERNS

- Text context Proj 1 - Dependency Injection
- All components in Proj 1 - Module Design Pattern
- UseState in Proj 1 - Hooks
- Components in Proj 2 - Module Design Pattern
- UseState and UseMemo, ApolloProvider, useQuery, useMutation in Proj 2 - Hooks
- Django backend in Proj 2 - MVC/MVT patterns??
- Sockets from Final Proj - Observer pattern
- Backend in Final Proj - MVC Pattern

React Components follow the module design pattern as they separate logic for reusability and scalability. React components also allow for private/public encapsulation meaning that unless we export our component, the declarations defined in our module will remain local to that module. I use react components throughout Project 1 and Project 2 on the basis of component thinking which inherently aligns with the Module Design Pattern. I also follow other design patterns in my projects such as dependency injection, React hooks for either React Context API, or our Apollo React hooks for querying and mutating the database. Lastly when developing the backend for the Final Project, I built off of the MVC design pattern when designing the file structure and

organization of our backend. Although I didn't implement the socket functionality, our group identified the functionality of group approvals to be an observer pattern in which all the members of a payment group subscribe to each other's approval status.

FROM project-1-personal-portfolio-Dragoon788/my-app/src/app/components/textBox.tsx, (images are also clickable)

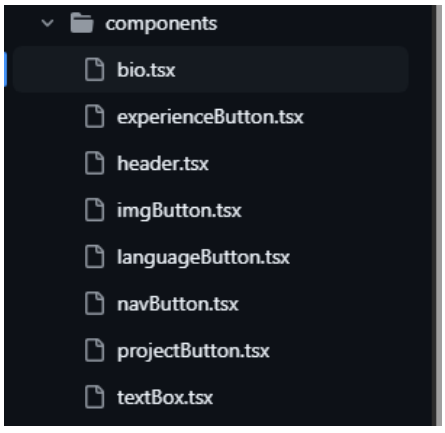
Code Artifact #6:

```
6  export default function Textbox() {
7      const { text, setText } = useTextContext();
8      const [input, setInput] = useState("");
9      return (
```

In Project 1, I create a textContext which is then imported in the textBox component demonstrating the Dependency Injection design pattern allowing for the abstraction of the textContext dependency through React Context API. This textContext is also an example of the Hooks Pattern based on React's custom hooks, which allow for sharing the context across different components with low coupling.

FROM project-1-personal-portfolio-Dragoon788/my-app/src/app/components

Code Artifact #7:




```
13  export default function Bio({ }: BioProps){
```

In Project 1, the components directory in my file structure follows the Module Design pattern as each component is separated by application logic allowing for optimal reusability and making the application more robust and easier to fix different components.

Code Artifact #8:

FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/FullPost.tsx

```
47  export default function FullPost({ postId }: FullPostProps) {
48    const [isEditing, setIsEditing] = useState(false);
49    const [editTitle, setEditTitle] = useState('');
50    const [editContent, setEditContent] = useState('');
51
```

In Project 2, I also utilize useState throughout my different components to manage the state. The useState hook allows us to leverage the component lifestyle in remounting our state with new updated information. In this example, I'm using the useState hook to remount whenever the state is updated making sure the display properly shows their updated post.

Code Artifact #9:

FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/apolloWrapper.tsx

```
18  export function ApolloWrapper({ children }: { children: React.ReactNode }) {
19    const client = useMemo(() => makeClient(), []);
20
21    return <ApolloProvider client={client}>{children}</ApolloProvider>;
22  }
```

The ApolloProvider that we use in Project 2 to access the database from is an example of the dependency injection pattern as it utilizes React's Context API to inject the Apollo Client

instance into our frontend. Following the dependency injection pattern, the Apollo Client is abstracted such that each component doesn't need to know how to set up the database access.

Code Artifact #10:

FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/CommentSection.tsx

```
64      const { data, loading, error } = useQuery<{ postComments: CommentType[] }>(GET_POST_COMMEN
65        variables: { postId },
66      });
67
```

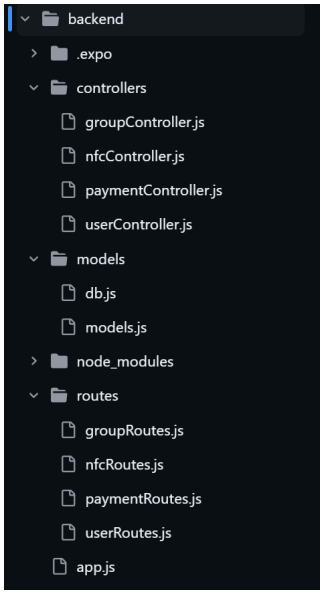
From our Apollo Client, we also import and use React hooks for accessing the database, 'useQuery and 'useMutation'. These React hooks follow the Hooks pattern.

Code Artifact #11:

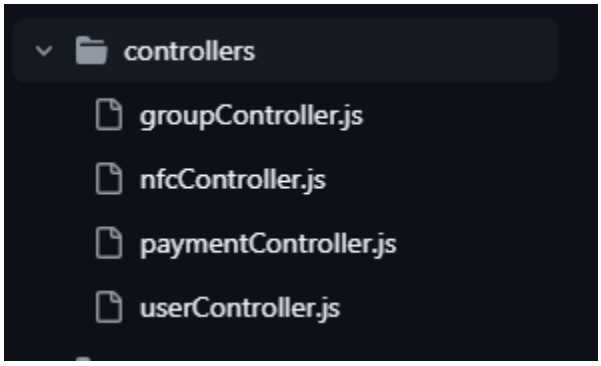
```
68      const [createComment] = useMutation(CREATE_COMMENT, {
69        refetchQueries: [
70          { query: GET_POST_COMMENTS, variables: { postId } },
71          { query: GET_USERS }, // Also refetch users after comment creation
72        ],
73      });
```

Code Artifact #12:

FROM final-project-figgystacked/backend



When designing the backend for the Final Project, I followed the MVC pattern in separating our routing logic, business logic, and database access in our file structure. My models directory handles all the database connection, setup, and defines our User, Group, and NFCCard models and relationships in our database. The controllers directory handles all the logic for querying and mutating our database at different API endpoints. The routes directory sets up all the API endpoints and connects them to the controllers. The view in this instance would be the frontend and how it fetches data from the backend endpoints following REST API.



Code Artifact #13:

FROM final-project-figgystacked/backend/controllers/groupController.js

```
39          // Broadcast to all users in the group room
40          const io = getIO();
41          io.to(`group-${groupId}`).emit("groupState", groupData);
42          console.log(`✅ Broadcasted group update for group ${groupId} to ${group.Users.length} members`);
```

Although I didn't write the code for creating and managing the websockets, our group had identified that our group approval process on the web app followed the Observer Pattern. The observers in this instance are each group member subscribing to each member's approval status. We needed this information to be shared by websockets because we wanted to properly update each user's view in real time.

FULL-STACK DESIGN PRINCIPLES

Project 1

- Component thinking
    - Reference the previous code artifacts
- Contexts and Prop handling
    - Text context in Proj 1

Project 2:

- Each model is clearly divided and focused on a particular app goal (high cohesion)
- Components directory (high cohesion)
- GraphQL resolvers in schema.py are grouped by entity
- Data fetching is abstracted using useQuery and useMutation hooks (low coupling)
- Components are not dependent on each other (low coupling)
    - Component thinking reference previous code artifacts
- ApolloProvider (low coupling)

Final Project:

- MVC pattern and clear interface interaction (low coupling)
    - Models are clearly defined and their roles are distinct, routes are distinct (high cohesion
- Sequelize abstracts querying and mutating data in the data base much like django (low coupling)

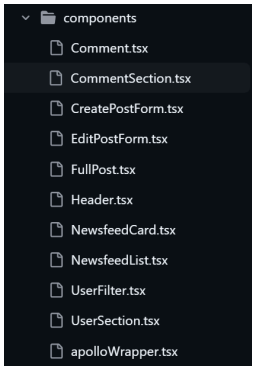
Project 1:

Throughout Project 1, my goal was to have high cohesion and low coupling. I accomplished this by incorporating a lot of component thinking principles. As demonstrated in Code Artifact #7, I created many components, each handling different logic and functionality – high cohesion in component architecture. The different components are also independent and don't rely on each other's logic to function – low coupling between components. In Project 1, I also created a textContext – shown in Code Artifact #6 – which abstracted the contents of my text box and allowed multiple components to access the text state – reducing tight coupling across components. The text context also groups all the related text information in a single context – high cohesion in the context component.

Code Artifact #14:

FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/



Project 2:

In Project 2, I structured the backend and frontend to achieve high cohesion and low coupling through distinct separations in GraphQL models and resolvers, component thinking, using query and mutation hooks, and using the ApolloProvider for client setup. Like in Project 1, I also highlighted component thinking principles in my frontend components directory in Code Artifact #14 – high cohesion and low coupling. Another example of how I balanced coupling and cohesion is shown in Code Artifact #12, where the models are clearly defined for the User, Group, and NFCCard separating the data and methods around our web app's main functionality – high cohesion. The way I interacted and used the Apollo client also demonstrates the balance between coupling and cohesion. The useQuery and useMutation hooks reduce coupling from the frontend and backend because the logic for calling upon the database is abstracted and decouples the frontend from data fetching logic shown in Code Artifact #10. Lastly, the ApolloProvider uses the React Context API to inject GraphQL client into the frontend reducing the coupling between the frontend and client setup logic.

Code Artifact #15:

FROM final-project-figgystacked/backend/controllers/models/models.js

```
106     Group.hasMany(User, {
107       foreignKey: 'groupID',
108       onDelete: 'SET NULL',
109       hooks: true
110     });
```

Final Project:

When I set up the backend of our Final Project, I followed the MVC pattern in my file structure which illustrates high cohesion between model logic and reduced coupling between the models, views, and controllers. Like in Project 2, the clearly defined models are built around different app features which helps ensure high cohesion between our models (shown in Code Artifact #15) and demonstrate how they work together to enable database access. The MVC pattern (shown in Code Artifact #12) also reduces the coupling between the models, views, controllers as their functionality is independent and can be tested, scaled, and worked on separately. This was extremely helpful in a collaborative project because my group could work on our backend and frontend simultaneously. Similarly to Project 2, I utilized sequelize to abstract data fetching and mutation which reduced coupling between the frontend and database access.

SYSTEM DESIGN/PERFORMANCE

- GraphQL api vs REST
- Django vs not
- Frontend:
    - Apollo caching improves perceived performance.
    - React's virtual DOM and hooks optimize UI updates.
- Backend:
    - Django's ORM is efficient for most CRUD, but for high-scale, you might need to optimize queries or use async frameworks.

    - GraphQL batching and query complexity can be tuned for performance.
- By separating backend logic into controllers, models, and routes, you gain maintainability and scalability, but at the cost of more files and initial complexity.
- By separating backend logic into controllers, models, and routes, you gain maintainability and scalability, but at the coIf you use socket.js in the frontend, you enable real-time communication, which improves user experience but adds complexity in state management and error handling.st of more files and initial complexity.

Project 1:

In Project 1, my system architecture revolved around the Module Design Pattern and Dependency Injection patterns. Using the Module Design pattern directly correlated with component thinking principles and separating the components into distinct functional units helped maintain code reusability and private/public encapsulation. On the other hand, the # components increased following these principles and led to more state management across different components and the necessity for a shared context. I believe it was still worth it because the state management issues across multiple components can easily be solved with React's context API. I also identified the dependency injection pattern in my TextContext which reduced the coupling between different components consuming that require the text content. Although the TextContext reduced tight coupling, I believe that it was not worth it to make a textContext in this instance since it was only used by one component and created unnecessary complexity.

Project 2:

In Project 2, my architecture and design patterns were influenced by the Module Design pattern and Hooks Pattern. Much like how the Module Design pattern influenced my project 1, the same benefits and downsides exist for this project. I believe that the Module Design pattern is overall

helpful in this project despite the complexity of working the # of components because the file structure organization and component reusability are essential parts in making scalable, maintainable code. This project also emphasized the importance of the Hooks Pattern in designing the system architecture. The main benefit of identifying the Hooks Pattern is the simplification of state management and the work around the component lifecycle; specifically in Project 2, the useQuery and useMutation hooks abstract data fetching and automatically handle loading, error, and response states. On the other hand, the useQuery and useMutation hooks create tight coupling between the component and the Apollo Client, making it more complex to initialize a test and decouple frontend logic from data fetching logic. This pattern was still overall helpful as we were instructed in our assignment to use the Apollo Client (I didn't intend to switch) and the Apollo Client made database access much easier and replicable across different components.

Final Project:

Lastly, my final project was influenced by MVC and Observer Patterns. Structuring the backend of our final Project in the MVC pattern helped

API FORMATS

I would say that my favorite API format that we learned about and got the chance to work with would be GraphQL APIs. Since I had only known about REST APIs going into class, I found the contrast between GraphQL and REST to highlight the benefits of GraphQL. I liked GraphQL a lot because I felt its query language was really intuitive and easy to understand and enjoyed the structured organization it provided for setting up the connection between the frontend and backend. The way it was presented in class it also seemed like GraphQL was a natural

progression of REST APIs, solving many of the issues that came with using REST (underfetching, overfetching, multiple endpoints) without really introducing any substantial tradeoffs. Although I've worked mostly with REST APIs, I think learning both now highlights the flexibility and benefits of using GraphQL in a modern Full-Stack environment.

AI TOOLING

I really enjoyed Prof. Tse's discussion and approach to us using AI in this class. I think she provided a really important learning experience into the way in which we can and should use AI in our future industry work. Learning to work with and properly balance AI's use in our projects was extremely helpful in determining the right ways to use AI to give us an advantage. Prof. Tse was really inspiring in describing how to use AI to stay ahead of competition and to become better developers rather than repeating the doom and gloom of AI taking over CS jobs. I feel this topic is rarely discussed from professors and getting the chance to hear how an industry professional views AI gives me hope for the future of AI in my career and the industry as a whole.

Problem Solving:

Project 1:

I had previous experience working on the frontend specifically from Prof. Tse's Web Development class, so I had a decent understanding of component thinking, working with Tailwind, and using Next.js. Although I had previous experience, I think properly setting up the page was my biggest technical challenge. Because I was designing the page based off of someone else's portfolio, I tried my best to recreate the structure from scratch and had to complete lots of research on how to mimic the design using Tailwind. One of these issues was keeping the left side of the screen stationary while the right side would be scrollable. This

involved looking through several Tailwind docs in order to properly find how to make the content of the screen half the page and also fixed. In general though, I think I struggled working with CSS because I only ever learned for the Web Development course and always struggled with flexbox and grid layouts.

Code Artifact #1:

```
13  export default function Bio({ }: BioProps){
14      const { text, setText } = useTextContext();
15      return(
16          <div className="w-1/2 bg-red-700 h-full fixed left-0 top-0 z-10 p-10">
17              <Header name="Francis Velasco"
18              title="Full Stack Software Engineer"
19              des="Northwestern Computer Science student with a degree soon!"
20              />
```

In project-1-personal-portfolio-Dragoon788/blob/main/my-app/src/app/components/bio.tsx, I used Tailwind CSS styling to fix the left hand side of the page, and allow it to not be scrollable.

Code Artifact #2:

```
16          <div className="flex flex-col p-5 ml-[50%] w-1/2 h-screen bg-green-700 p-10">
17            <nav className="flex justify-center m-10">
```

In project-1-personal-portfolio-Dragoon788/blob/main/my-app/src/app/page.tsx, I wrap the right side of the page in a flexbox, push it to the right hand side of the page, and initialize the margins. I also justify the nav bar in the center of that div container.

Project 2:

One major issue that I overcame in Project 2 was fixing hydration errors when trying to initialize contexts with response information from our queries. I thought this would be especially helpful for information that was shared across multiple components like the list of comments or the list of posts. When I tried to initialize the contexts, I kept receiving Hydration Errors saying that my queries and mutations were called during page rendering – even though I had initialized the

context to use client side rendering. I eventually overcame this issue by dropping the contexts as a whole and having each component query the database and store its response locally. This solution ensured that as long as the component used CSR, then the data would be fetched client-side. Although this may not have been the best solution, this problem couldn't be easily solved with AI or a google search. My debugging process involved isolating different pieces of my code to determine which code was causing the hydration issue. I eventually figured out it was my querying and like I said above, migrated the data fetching into each component.

Code Artifact #3:

```
8   const GET_USERS = gql`
9     query GetUsers {
10      users {
11        id
12        username
13      }
14    }
```

In project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/NewsfeedList.tsx, I call a GET_USERS query in the NewsfeedList component itself because the context provider was causing issues in the layout.tsx file when initially trying to set up the provider. This query call is replicated in other code artifacts below.

Code Artifact #4:

```
7   const GET_USERS = gql`
8     query GetUsers {
9       users {
10        id
11        username
12        email
```

In project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/CommentSection.tsx

Just like in Code Artifact #3, I also call the GET_USERS query to access all the users in my CommentSection component because it was causing hydration errors previously when trying to set up the context provider in layout.tsx.

Code Artifact #5:

```
17    const GET_POSTS = gql`
18      query GetPosts {
19        posts {
20          id
21          title
22          content
23          author {
24            username
```

In project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/NewsfeedList.tsx, I call a GET_POSTS query within the NewsfeedList component because it was causing hydration errors previously when trying to setup the context provider.

Final Project:

The major technical challenge that I faced when working on the backend was initializing the file structure using node and express.js and understanding how the pieces fit into the overall MVC architecture. Because this was my first ever time working on the backend, this seemingly easy challenge took a lot of reading through documentation, watching tutorials, and using AI as a tool to understand each file's importance in the backend. I began first by watching and following tutorials on setting up a project using express.js and using the videos as a boiler plate to work off and expand. This is where I learned to set up the file structure with routes, controllers, and models (shown in Code Artifact #14), as well as initialize the express server.

Code Artifact #6:

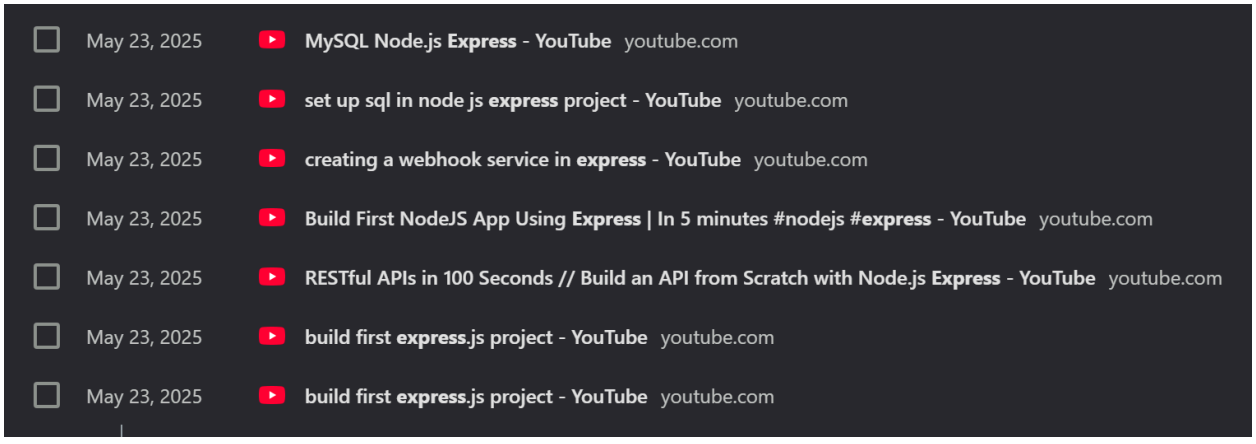
FROM

project-2-newsfeed-Dragoon788/.github/newsfeed_frontend/newsfeed-frontend/src/app/components/NewsfeedList.tsx

```
const express = require("express");
const http = require("http");
const { Server: ioServer } = require("socket.io");
const app = express();
// const router = express.Router();
const { sequelize, connectToDb } = require("./models/db");
const { Group, User } = require("./models/models");
const groupRoutes = require("./routes/groupRoutes");
const userRoutes = require("./routes/userRoutes");
const paymentRoutes = require("./routes/paymentRoutes");
const nfcRoutes = require("./routes/nfcRoutes");
const PORT = 3001;
```

This is my expanded boilerplate code in final-project-figgystacked/app.js that I based off of tutorials from YouTube.

Code Artifact #7:

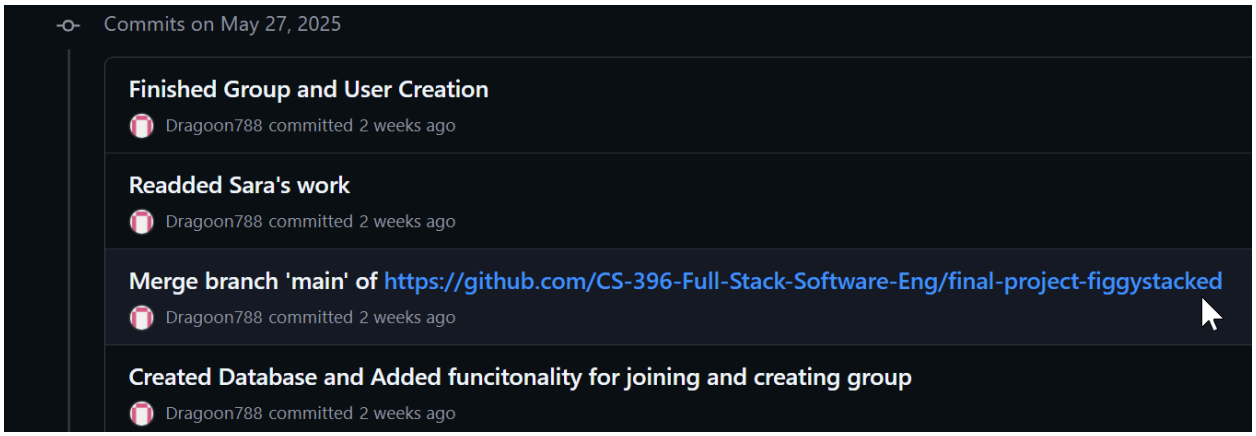


This is my YouTube History from when I was figuring out how to initialize our backend because it was my first time working with express.js and the backend as a whole.

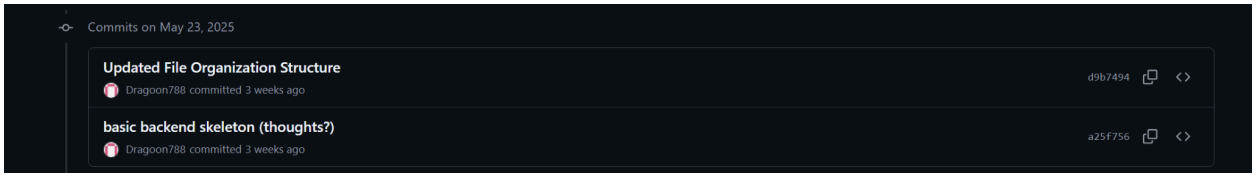
Collaboration:

a. For the Final Project I mainly contributed to the backend, working on the initialization of the database, connection to db, creating the file structure, setting up the node and express servers, and setting up the boiler plate for all our files. Working on the backend also included creating the functionality for creating a user, creating a group, joining a group, and deleting a group. I also worked on the design docs with the group members in mapping the overall architecture of our project and later on explaining how the backend interacts with the frontend.

Code Artifact #7:

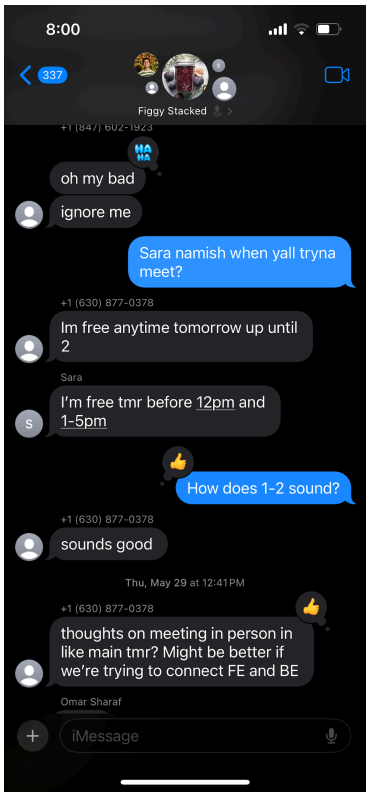


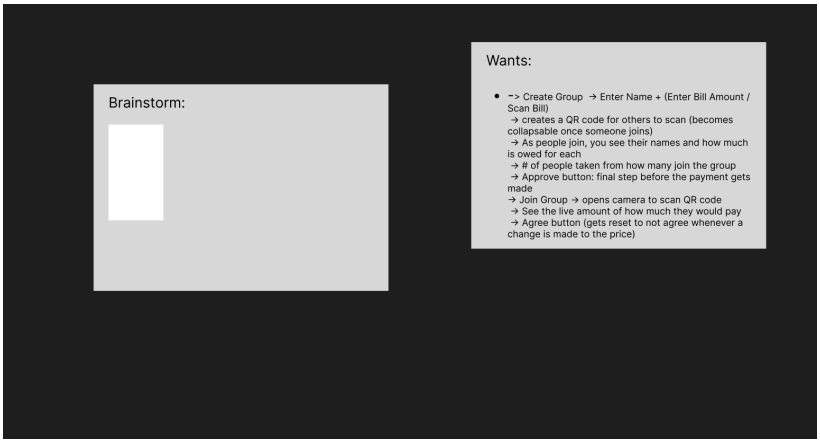
Code Artifact #8:

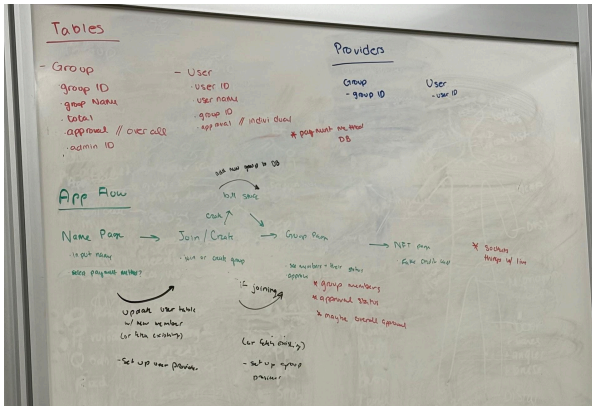


b. For my group, I was also the individual to start creating the figma document to arrange our ideas as well as being the go-to person to explain what was going on in the backend to Namish and Sara (who were also working on the backend). In our group meetings when we eventually wanted to connect the frontend to the backend, I explained the key points of our backend and how the frontend could fetch from the API endpoints. I worked with Omar to diagram our project and highlight what elements of our backend corresponded to code in our frontend.

Code Artifact #3, #4, and #5:







ACCOUNTABILITY AND GROWTH:

a. Although I didn't submit my projects before the feedback deadline, I made sure to submit each assignment before the final deadline without asking for extensions. I will take this into account when considering my grade, but also I tried my best to get through my work (even as a senior with lots of busy plans coming up) and I submitted before the cutoff deadline.

b. Throughout the course, I feel that I've built off of my previous experience in CS 396 Web Dev and have been able to challenge myself to learning new things that I didn't previously have time for or have less experience with. I feel that I've excelled at working with the backend (as someone with no previous experience). For the second and final project, I was exposed to backend development for the first time and had gotten to experience two different backend designs in both projects. In general, I think I've also excelled at learning how to use AI to help learn and write code. Often we're just told not to use AI at all, but I feel that Prof. Tse's discussion of AI helped me learn productive ways to use AI to excel in industry. I am really grateful for this open approach to AI, because it helped me figure out how to properly balance AI in my workflow.

   On things that I still need to improve, I think I definitely need to spend more time

working with CSS and Tailwind, as I've always struggled with flexbox and grid positioning and had to review it again this class (despite already learning it from the Web Dev course). As seen in Code Artifact #1 from problem solving, my positioning switches between Grid and Flexbox because I never understand either and I would definitely love to spend more time learning it.

Lastly, I would still be curious to learn about new ways in which the MV* model is being improved and the ways in which data streaming is being used for new technologies. I'm in general wanting to learn more about what things are new in software development and how those things get adopted and could be used in the future.

c. Throughout the projects for the quarter I completed different stretch goals. Although these challenges were not required for submission, I wanted to challenge myself and build off my previous Web Dev experience. You can see one of the stretch criteria for Project 1 involved using Dependency Injection. In this project, I created a textContext using React context API as seen in Code Artifact #6 in the Technical Skills section. I also challenged myself by working on the backend knowing that I had no previous experience with backend development.

Images for the code artifacts are clickable.

Code Artifact #1 (Create Comment Functionality on Backend):

```
30    const CREATE_COMMENT = gql`
31      mutation CreateComment($postId: ID!, $content: String!, $username: String!) {
32        addComment(postID: $postId, content: $content, author_username: $username) {
33          id
34          content
35          author {
36            username
37          }
38          created_at
39        }
40      }
41    `;
```

Code Artifact #2 (Submit Comment Functionality on Frontend):

```
75      const handleSubmitComment = async (e: React.FormEvent) => {
76        e.preventDefault();
```

Code Artifact #3 (UserFilter functionality on Frontend for Project 2):

```
21  export default function UserFilter({ selectedUser, onUserSelect }: UserFilterProps) {
22    const { data: userData, loading: userLoading, error: userError } = useQuery(GET_USERS, {
23      fetchPolicy: 'network-only',
24    });
25
```

Code Artifact #4:

```
92      // Get the posts array from the appropriate query result
93      const posts = selectedUser && data
94        ? (data as UserPostsData).getUsersPosts
95        : !selectedUser && data
96          ? (data as AllPostsData).posts
97          : [];
```

Final Grade:

Based on my reflection and evidence, I would assign myself an A. Although there are certainly aspects of the course I know/wish I could improve on, I definitely think that the grade should reflect the effort put into the work and results. Throughout this quarter, I had to balance working with very little experience, but regardless I produced results that I am personally very proud of and am glad I've gotten to improve on these skills for the future. So based on the effort put into this course in Lecture, in Projects, and in Group activities and my project results reaching all the criteria of the spec sheets, I believe that I deserve an A.